B-SyncText — Rapport de test

Kengfack Tsafack Bernice Lagrange

26 octobre 2026

Résumé

Ce rapport décrit comment B-SyncText a été testé sur AWS (région eu-north-1), les objectifs chiffrés fixés avant les tests, les résultats bruts et leur analyse. Il apporte la preuve que les deux contraintes les plus importantes du sujet ont été respectées : **un lien ne délivre jamais plus de N téléchargements** (contrainte 1) et **aucune modification acceptée n'est perdue, tous les rédacteurs voient le même document** (contrainte 2). Tous les fichiers de résultats bruts cités sont fournis avec le rapport.

Table des matières

1 Méthode **2**

1.1 Principes 2

1.2 Outils 2

2 Objectifs fixés avant les tests **3**

3 Résultats bruts **3**

3.1 T-01 — Exactement N téléchargements (contrainte 1) 3

3.2 T-02 — Remise en stock 3

3.3 T-03 — Aucune perte, même document pour tous (contrainte 2) 4

3.4 T-04 — Le document tel qu'il est au moment du clic (contrainte 3) 4

3.5 T-05 — Crédit journalier (contrainte 4) 4

3.6 T-06 et T-07 — Le pic (contraintes 5 et 6) 4

3.7 T-09 — Un seul e-mail par lien épuisé 5

3.8 T-08 — Blocage d'un rédacteur qui insiste 6

4 Analyse **6**

4.1 Preuve de la contrainte 1 6

4.2 Preuve de la contrainte 2 6

4.3 Ce que le pic a appris 6

4.4 Limites 7

4.5 Perspectives 7

5 Matrice de traçabilité **7**

1 Méthode

1.1 Principes

— **Les tests visent l'application déployée** : les garanties des contraintes 1 et 4 reposent sur les écritures conditionnelles de DynamoDB, et seul le vrai service peut les prouver.
— **Trois niveaux** : tests unitaires (Vitest, 112 tests, exécutés à chaque construction) ; tests de concurrence (scripts Node.js, un par contrainte) ; test de charge (k6) reproduisant le profil du sujet.
— **Chaque essai produit un fichier brut** (JSON daté) avec ses paramètres, ses mesures et son verdict, calculé par le script et non à la main.
— **Le générateur de charge tourne dans la région AWS**, sur une machine temporaire pilotée sans SSH : depuis un ordinateur personnel, la connexion ne tient pas mille connexions simultanées et fausse les mesures (le premier essai, lancé depuis le poste, a échoué pour cette raison : 90 % des demandes n'atteignaient pas AWS).
— Les mesures côté serveur viennent de CloudWatch (Lambda, API Gateway, EC2) et des journaux.

1.2 Outils

Outil	Usage
Vitest	tests unitaires TypeScript ; couverture de `src/commun` : 98 % des lignes (objectif 80 %)
Scripts Node.js (`tests/concurrence/`)	clients HTTP et clients Yjs simulés : T-01 à T-05, T-08, T-09
k6	charge (T-06, T-07) : arrivées programmées, mesures de temps de réponse, seuils
CloudWatch	métriques Lambda, API Gateway, EC2 ; journaux des fonctions
Générateur `m7i-flex.large`	machine temporaire dans eu-north-1, créée et supprimée par script

2 Objectifs fixés avant les tests

Test	Contrainte	Réussi si
T-01	C-1	1 000 demandes simultanées sur un lien de N (N = 1 et N = 500), 5 répétitions : exactement N PDF, numérotés de 1 à N sans doublon ; ticket rejoué refusé ; demandes suivantes « épuisé »
T-02	C-1 (remise en stock)	10 exemplaires réservés puis abandonnés : après 60 s, servis aux visiteurs qui patientent ; total 10
T-03	C-2	50 rédacteurs au même endroit, serveur redémarré pendant l'essai : textes identiques chez tous, chaque insertion confirmée présente, convergence < 5 s
T-04	C-3	modification puis téléchargement : le PDF contient la modification et son numéro de version
T-05	C-4	3 appareils du même rédacteur : jamais plus de 10 000 caractères acceptés
T-06	C-5	15 000 visiteurs (12 000 dans la 1re minute) et 300 rédacteurs : 100 % des visiteurs reçoivent un état ; p95 < 2 s ; 0 erreur serveur ; les rédacteurs continuent d'écrire
T-07	C-6	visiteurs en attente informés de l'épuisement en moins de 5 s
T-08	abus	blocage après 20 refus en une minute
T-09	e-mail	exactement un e-mail par lien épuisé

3 Résultats bruts

Fichiers dans `resultats/` (un JSON par essai, nommé `<test>-<date>.json`).

3.1 T-01 — Exactement N téléchargements (contrainte 1)

Fichier : `t01-c1-20261025-155146.json`. 1 000 visiteurs simultanés par essai, depuis le générateur.

N	PDF reçus (5 essais)	Manquants	Doublons	Ticket rejoué	Durée des essais (s)
1	1, 1, 1, 1, 1	0	0	refusé (410)	5,7 ; 3,0 ; 2,3 ; 2,3 ; 2,5
500	500, 500, 500, 500, 500	0	0	refusé (410)	63,6 ; 3,3 ; 65,3 ; 7,8 ; 3,1

Verdict : **réussi**. Sur 10 essais, le nombre de PDF délivrés est exactement N, les numéros d'exemplaire vont de 1 à N sans trou ni doublon, 100 demandes faites après l'épuisement reçoivent « épuisé ». Les deux essais de 64 s à N = 500 montrent la remise en stock : des visiteurs simulés trop lents ont laissé expirer leur réservation (60 s), et leurs exemplaires ont été servis à d'autres.

3.2 T-02 — Remise en stock

Fichier : `t02-remise-en-stock-20261025-155549.json`. 10 réservations abandonnées, 20 visiteurs qui patientent : les 10 exemplaires sont servis 61,4 s après le début, les 10 autres visiteurs voient « épuisé », les anciens tickets ne servent pas. Verdict : **réussi**.

3.3 T-03 — Aucune perte, même document pour tous (contrainte 2)

Deux essais, 50 clients Yjs écrivant au même endroit, chaque insertion portant un marqueur unique. Fichiers : `t03-c2-20261025-155444.json` et `t03-c2-20261025-160218.json`.

Essai	Insertions confirmées	Manquantes	Textes identiques	Convergence
60 s, sans incident (fichier …`-155444`)	7 608	0	50/50 + client neuf	78 ms
150 s, **serveur redémarré** à 16 :00 :24 UTC (fichier …`-160218`)	18 691	0	50/50 + client neuf	214 ms

Verdict : **réussi**. Dans le second essai, le serveur d'édition a été redémarré (`systemctl restart`) alors que les 50 connexions étaient ouvertes : les 18 691 insertions confirmées sont toutes présentes, les 50 clients et un client neuf (qui reconstruit le document depuis S3 et le journal DynamoDB) ont la même empreinte SHA-256 du texte.

3.4 T-04 — Le document tel qu'il est au moment du clic (contrainte 3)

Fichier : `t04-c3-20261025-225833.json` (mode « temps réel »). Un lien de 3 exemplaires est créé ; un premier PDF ne contient pas le marqueur ; le marqueur est inséré ; le second PDF, téléchargé 0,5 s après, le contient, avec « Version v0.2 · Édité le 25/10/2026 à 23 :58 » et « Exemplaire 2 / 3 ». Verdict : **réussi**.

3.5 T-05 — Crédit journalier (contrainte 4)

Fichier : `t05-c4-20261025-155445.json`. 3 appareils du même rédacteur écrivent en parallèle par blocs de 200 caractères : 10 000 caractères acceptés exactement, crédit restant 0 côté serveur, un quatrième appareil refusé. Verdict : **réussi**.

3.6 T-06 et T-07 — Le pic (contraintes 5 et 6)

Premier essai (25/10, 23 :13 UTC) Fichiers dans `resultats/t06-essai1/`.

- 144 197 requêtes en 5 min (480/s) ; temps de réponse p95 = **193 ms**, maximum 1,44 s.
- **500 PDF livrés exactement** (lien : comptés 500, numérotation 500, disponibles 0, état épuisé) ; 9 775 visiteurs ont reçu « épuisé ».
- Visiteurs en attente informés de l'épuisement entre +0,0 s et **+4,2 s** (T-07 : objectif < 5 s).
- **Défaut** : 611 saturations Lambda (concurrence réservée de 250 atteinte), transformées en erreurs 5xx par API Gateway (0,42 % des requêtes) ; 17 visiteurs simulés ont dépassé leur délai de réessai sans état. Cause : au premier instant, 500 visiteurs « prêts » demandent le même PDF qui n'existe pas encore, et chacun le génère.
- Le script des 300 rédacteurs a planté à la fin (dépassement de pile dans le calcul d'un maximum) : pas de mesure de latence d'édition pour cet essai. CPU du serveur d'édition : 58 % au maximum.

Corrections : une seule génération de PDF par version (verrou de 20 s sur le document, les autres exécutions attendent le fichier) ; concurrence réservée portée à 300 ; script corrigé.

Second essai (23 :26 UTC) Fichiers dans `resultats/t06-essai2/`. Après le verrou de génération et la concurrence réservée de 300 : 144 813 requêtes, p95 = 131 ms, 500 PDF exactement, visiteurs en attente informés de l'épuisement en 1,1 s (médiane) et 3,5 s au maximum ; mais **encore 613 saturations** (concurrence 300 atteinte) et 6 visiteurs sans état. Explication : le verrou évite les générations multiples, mais les 499 exécutions qui attendent le fichier occupent chacune une

exécution Lambda pendant une seconde. Les 300 rédacteurs, mesurés cette fois : 143 569 insertions envoyées, 82 048 confirmées, latence p95 de **99 s** : le serveur d'édition saturait un cœur (CPU 58 % des 2 vCPU), sans aucune limitation DynamoDB.

Corrections : le PDF est **préparé à la demande d'exemplaire**, avant la réservation (un seul visiteur le génère, les autres reçoivent « patientez » une seconde) ; dans le serveur d'édition, suppression d'un instantané Yjs du document entier pris à chaque modification pour détecter les renvois (remplacé par une comparaison d'horloges, coût proportionnel à la modification).

Troisième essai (23 :38 UTC) Fichiers dans `resultats/t06-essai3/`. Visiteurs : **0 erreur 5xx, 0 itération abandonnée**, 27 437 visiteurs servis, p95 = **31 ms** (maximum 1,46 s), 500 PDF exactement, 42 réponses « patientez » seulement, visiteurs en attente informés en **2,0 s** au maximum. Le profil visiteurs du sujet est tenu. Rédacteurs : toujours en échec (p95 102 s), le cœur Node restant saturé : seconde cause trouvée, la longueur du texte (contrôle des 200 000 caractères) était recalculée en parcourant tout le document à chaque modification acceptée. Correction : longueur tenue à jour par addition, recalcul exact seulement à l'approche de la limite.

Quatrième essai (23 :49 UTC) Fichiers dans `resultats/t06-essai4/`. Visiteurs : à nouveau **0 erreur, 0 itération abandonnée**, 28 819 visiteurs servis, p95 = **30 ms**, 500 PDF exactement, visiteurs en attente informés en **2,0 s** au maximum. Rédacteurs : 92 110 insertions confirmées sur 143 398 envoyées, latence p95 de 85 s : mieux, mais le serveur d'édition reste saturé à ce rythme (environ 400 modifications par seconde traitées pour 600 envoyées). Mesure de capacité, 300 rédacteurs sur 30 documents (trois processus clients de 100, 90 s chacun, fichiers dans `resultats/t06-capacite-redacteurs/`) :

Rythme par rédacteur	Modifications/s	Confirmées	Latence p95	Verdict
2 insertions/s	600	≈ 67 %	41 s	**échec** (saturation du cœur Node)
1 insertion/s	300	100 % (27 021/27 021)	**101 ms**	**réussi**
0,5 insertion/s	150	100 % (13 501/13 501)	25 ms	**réussi**

Le serveur d'édition unique (t4g.small) tient donc 300 rédacteurs à un rythme de frappe soutenu (une insertion par seconde chacun, 300 modifications par seconde, latence p95 de 100 ms) et plafonne vers 400 modifications par seconde. L'objectif de latence (< 1 s) est tenu jusqu'à ce plafond.

Bilan du pic Le profil visiteurs du sujet (15 000 clics, 80 % dans la première minute) est tenu avec une large marge : chaque visiteur reçoit un état, p95 de 30 ms, aucune erreur, exactement N exemplaires, épuisement annoncé en 2 s. Le point faible mesuré est le débit d'édition du serveur unique (t4g.small) quand 300 rédacteurs envoient chacun 2 modifications par seconde ; le sujet ne fixe pas ce rythme, et la section suivante donne le débit réellement tenu.

3.7 T-09 — Un seul e-mail par lien épuisé

Fichier : `t09-email-20261025-235615.json`. Lien d'un exemplaire créé avec une adresse vérifiée : demande « prêt », fichier reçu (200), puis en insistant : « épuisé », ticket rejoué refusé (410), « épuisé ». Le lien porte l'état « e-mail envoyé » et le compteur d'envois de SES a augmenté de 1 exactement. Verdict : **réussi**.

3.8 T-08 — Blocage d'un rédacteur qui insiste

Fichier : `t08/t08-blocage-*.json`. Un rédacteur épuise son crédit (10 000 caractères acceptés), puis insiste : au vingtième refus en moins d'une minute, il est bloqué (date de fin posée, écritures refusées, lecture seule), et figure dans la liste des personnes bloquées du tableau de bord. Verdict : **réussi**.

4 Analyse

4.1 Preuve de la contrainte 1

Le mécanisme est une **écriture conditionnelle atomique** dans DynamoDB : réserver un exemplaire, c'est exécuter « `disponibles = disponibles - 1` si `disponibles > 0` » ; DynamoDB applique cette instruction d'un bloc, deux demandes simultanées ne peuvent pas obtenir le même exemplaire. Le fichier n'est servi qu'avec un ticket à usage unique, consommé par une seconde écriture conditionnelle. La preuve : T-01 (10 essais, 10 000 demandes simultanées en tout) et le pic (20 000 visiteurs) donnent **toujours exactement N PDF**, numérotés sans doublon, et le compteur du lien dans la base est cohérent (`comptés = numérotation = N`).

4.2 Preuve de la contrainte 2

Le document est un CRDT (Yjs) : les modifications concurrentes se fusionnent dans n'importe quel ordre vers le même texte. Une modification n'est **acceptée** qu'après son écriture dans le journal DynamoDB ; elle n'est diffusée qu'ensuite. La preuve : T-03 avec redémarrage du serveur en pleine écriture, 18 691 insertions, 0 perdue, empreintes identiques sur 51 clients.

4.3 Ce que le pic a appris

Quatre essais ont été nécessaires, et c'est le résultat le plus utile de ce rapport : chaque essai a révélé une cause précise, corrigée puis remesurée.

1. Une fonction Lambda saturée ne donne pas un 429 au visiteur mais une erreur 5xx d'API Gateway ; la page réessaie dans les deux cas, mais l'objectif « 0 erreur serveur » impose d'éviter la saturation elle-même.
2. Générer le même PDF dans des centaines d'exécutions en même temps (le « stampede » du premier instant) : un verrou ne suffit pas, car les exécutions qui attendent occupent la concurrence ; il faut préparer le fichier avant de laisser les visiteurs le demander.
3. Dans le serveur d'édition, toute opération dont le coût croît avec la taille du document (instantané, recomptage de la longueur) devient prohibitive à plusieurs centaines de modifications par seconde : le coût par modification doit dépendre de la modification seule.
4. Les transactions DynamoDB sont annulées quand des centaines d'écritures visent la même fiche : le premier essai de T-01 a compté des exemplaires sans les livrer. Chaque étape (réserver, compter, remettre en stock) est donc une écriture conditionnelle simple, réessayée en cas de saturation ; en cas de panne entre deux étapes, le pire est un exemplaire perdu, jamais un exemplaire de trop.

Le premier essai a révélé un comportement d'AWS que la conception n'avait pas anticipé : une fonction Lambda saturée ne donne pas un 429 au visiteur mais une erreur 5xx d'API Gateway. La page publique réessaie dans les deux cas, mais l'objectif « 0 erreur serveur » n'était pas tenu.

4.4 Limites

— k6 a lancé environ 20 000 itérations au lieu de 15 000 (plus 4 707 abandonnées faute de VU libres sur le générateur) : l'essai était plus dur que le profil du sujet.
— Le serveur d'édition est unique : en cas de panne, l'édition s'interrompt le temps du redémarrage, sans perte (T-03).
— Les e-mails sont limités par le mode d'essai de SES (adresses vérifiées seulement).

4.5 Perspectives

— **Plusieurs serveurs d'édition**, avec une diffusion partagée entre eux, si le nombre de rédacteurs dépassait quelques centaines : les mesures de capacité montrent un plafond vers 400 modifications par seconde pour un seul serveur.
— **Un serveur d'édition plus gros** serait la première étape, car elle ne demande qu'une ligne d'infrastructure.
— **Sortie du mode d'essai du service d'e-mail**, pour que T-09 puisse envoyer à n'importe quelle adresse.
— **Rejouer le pic** sur plusieurs générateurs de charge, pour atteindre exactement le profil du sujet sans itérations abandonnées.

5 Matrice de traçabilité

Contrainte	Mécanisme	Test	Résultat	Preuve
C-1	décrément conditionnel, ticket à usage unique	T-01, T-02, T-06	10 essais + 4 pics : toujours exactement N, 0 doublon	`t01-*.json`, `t06-essai*/`
C-2	Yjs, acceptée = écrite au journal avant diffusion	T-03	18 691/18 691, convergence 214 ms	`t03-*.json`
C-3	état reconstruit au clic, version numérotée	T-04	marqueur présent	`t04-*.json`
C-4	crédit par jour, décrément conditionnel	T-05	10 000/10 000, 4e appareil refusé	`t05-*.json`
C-5	chemins séparés, PDF préparé à la demande, réessai côté page	T-06	28 819 visiteurs, 100 % avec un état, p95 30 ms, 0 erreur ; 300 rédacteurs à 300 modif./s, p95 101 ms	`t06-essai4/`, `t06-capacite-redacteurs/`
C-6	état « épuisé » définitif, mis en cache	T-07	visiteurs en attente informés en 2,0 s au plus	`t06-essai4/`